

Hybrid Computing Technology

1

CPU-Based Computing Technology

ผศ. อัครินทร์ คุณกิตติ

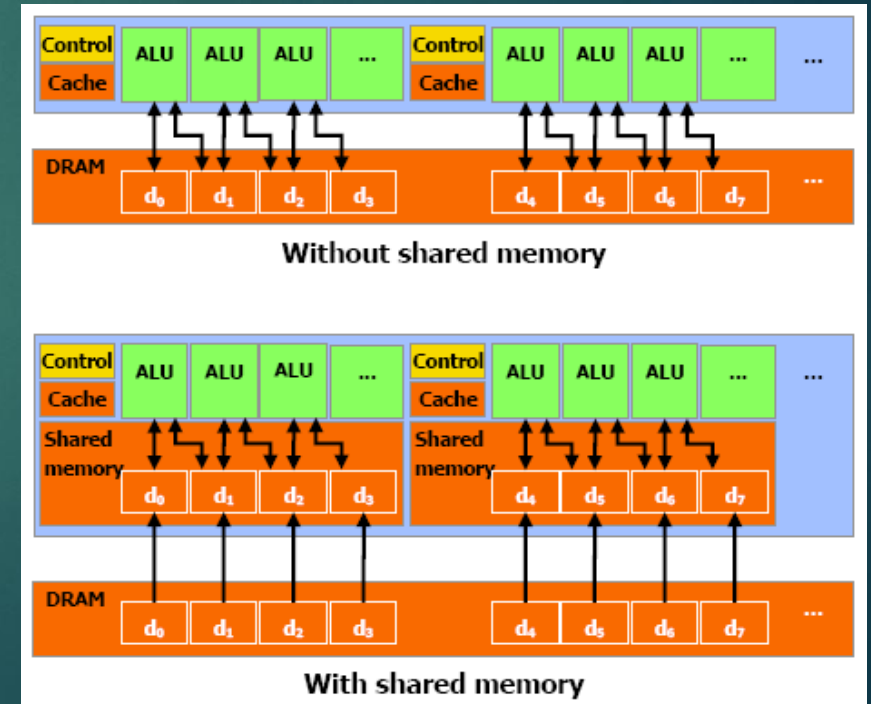
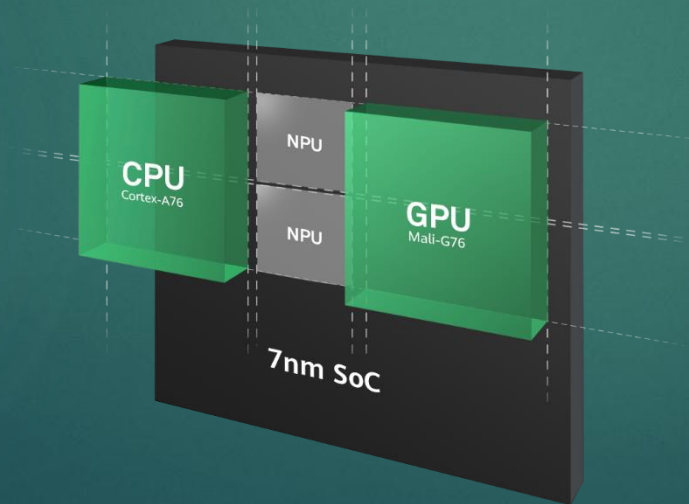
ASST. PROF. AKHARIN KHUNKITTI

FACULTY OF INFORMATION TECHNOLOGY
KMITL

Topics

2

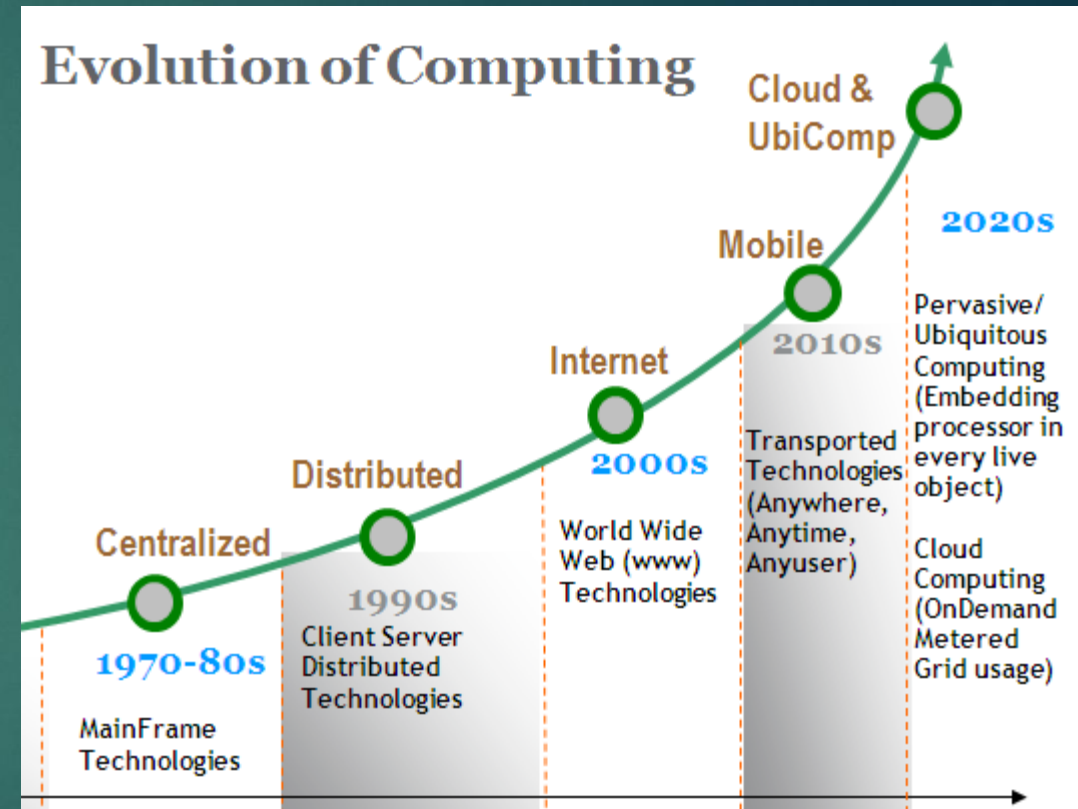
- ▶ CPU Technology and Performance
- ▶ Parallel Processing Architecture
- ▶ High Performance Computing in Processor
- ▶ Conclusion



Supercomputing Transition

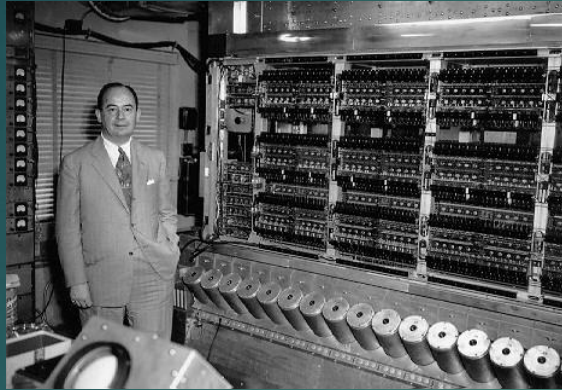
3

- ▶ Automated calculating => 17th century
- ▶ Stored program digital electronic => 1948
- ▶ Vector => 1975
- ▶ SIMD => 1980s
- ▶ MPPs => 1991
- ▶ Commodity Clusters => 1993/4
- ▶ Multicore => 2006



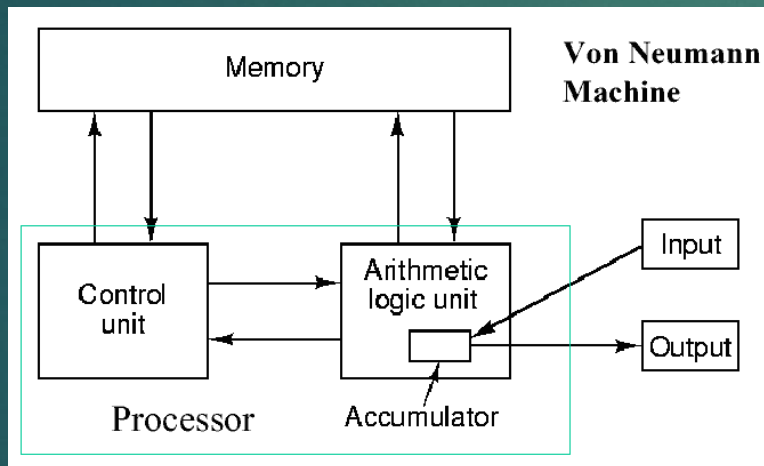
The von Neumann Computer

4



Walk-Through: $c=a+b$

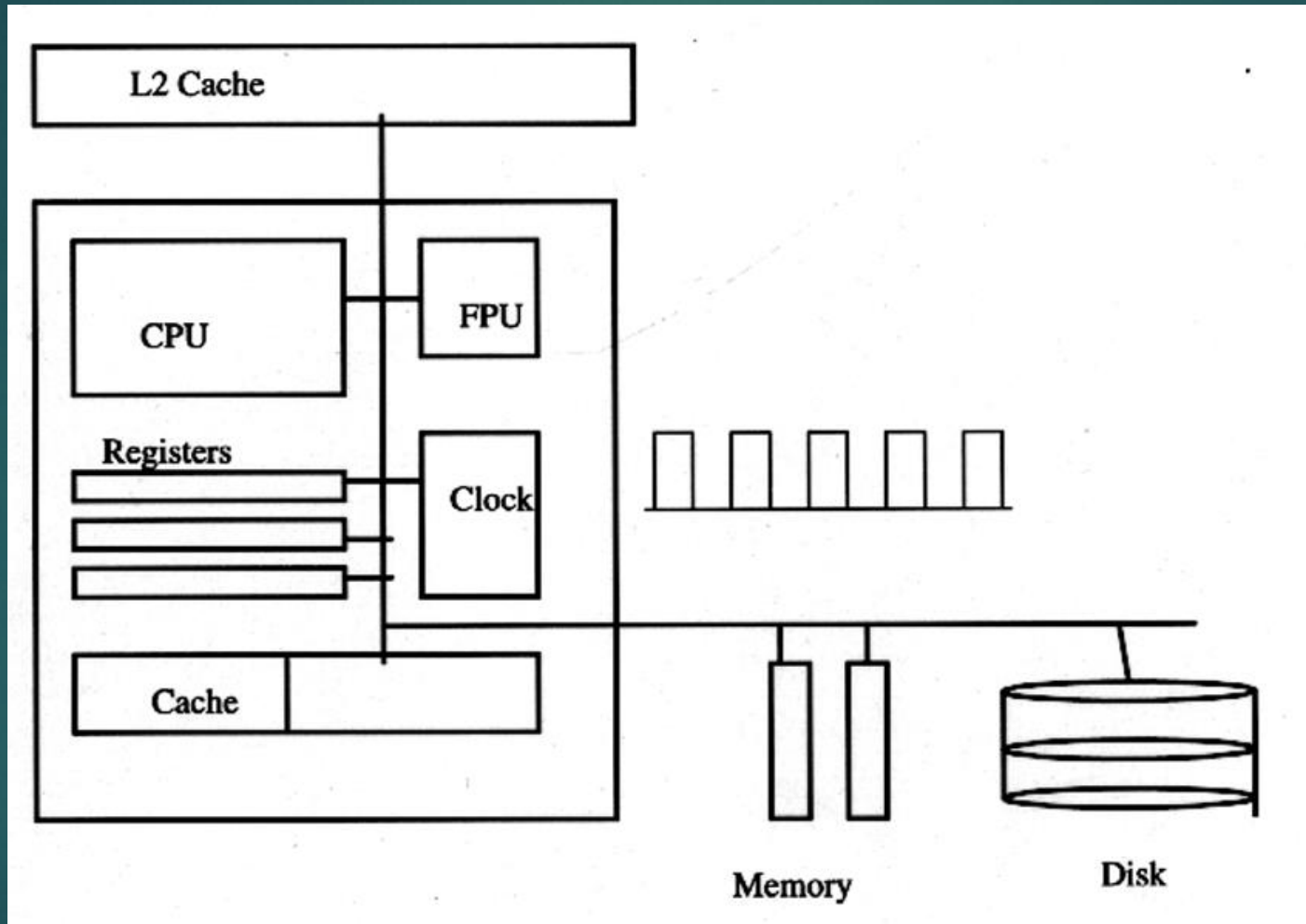
1. Get next instruction
2. Decode: Fetch a
3. Fetch a to internal register
4. Get next instruction
5. Decode: fetch b
6. Fetch b to internal register
7. Get next instruction
8. Decode: add a and b (c in register)
9. Do the addition in ALU
10. Get next instruction
11. Decode: store c in main memory
12. Move c from internal register to main memory



Note: *Some units are idle while others are working...waste of cycles.
Pipelining (modularization) & Caching (advance decoding)...parallelism*

Basic Architecture

5



- CPU, pipelining
- Memory hierarchy, cache

Computer Performance

6

- ▶ CPU operates on data. If no data, CPU has to wait; performance degrades.
 - ▶ typical workstation: 3.2GHz CPU, Memory 667MHz. Memory 5 times slower.
 - ▶ Moore's law: CPU speed doubles every 18 months
 - ▶ Memory speed increases much much slower;
- ▶ Fast CPU requires sufficiently **fast** memory.
- ▶ Rule of thumb: Memory size and Operations
 - ▶ 1CPU cycle (1 FLOPS) handles 1 byte of data
 - ▶ 1MFLOPS needs 1MB of data/memory
 - ▶ 1GFLOPS needs 1GB of data/memory

Many “tricks” designed for performance improvement targets the memory

CPU Performance

7

- ▶ Computer time is measured in terms of CPU cycles
 - ▶ Minimum time to execute 1 instruction is 1 CPU cycle
- ▶ Time to execute a given program:

$$T = n_c \times t_c = n_i \times \frac{n_c}{n_i} \times t_c = n_i \times CPI \times t_c$$

n_c : total number of CPU cycles

n_i : total number of instructions

$CPI = n_c/n_i$, average cycles per instruction

t_c : cycle time, 1GHz $\rightarrow t_c = 1/(10^9 \text{Hz}) = 10^{-9} \text{sec} = 1 \text{ns}$

To Make a Program/Computer Faster...

8

- ▶ Reduce cycle time t_c :
 - ▶ Increase clock frequency; however, there is a physical limit
 - ▶ In 1ns, light travels 30cm
 - ▶ Currently ~ GHz; 3GHz cpu $\rightarrow$ light travels 10cm within 1 cpu cycle $\rightarrow$ length/size must be < 10 cm.
 - ▶ 1 atom about 0.2 nm;
- ▶ Reduce number of instructions n_i :
 - ▶ More efficient algorithms
 - ▶ Better compilers
- ▶ Reduce CPI (Increase IPC – Instructions Per Cycle) -- The key is parallelism.
 - ▶ Instruction-level parallelism. **Pipelining** technology
 - ▶ Internal parallelism, multiple functional units; **superscalar** processors; multi-core processors
 - ▶ External parallelism, multiple CPUs, **parallel** machine

Processor Types

9

- ▶ Vector processor;
 - ▶ Cray X1/T90; NEC SX#; Japan Earth Simulator; Early Cray machines; Japan Life Simulator (hybrid)
- ▶ Scalar processor
 - ▶ CISC: Complex Instruction Set Computer
 - ▶ Intel 80x86 (IA32)
 - ▶ **RISC: Reduced Instruction Set Computer**
 - ▶ **Sun SPARC, IBM Power #, SGI MIPS**
 - ▶ VLIW: Very Long Instruction Word; Explicitly parallel instruction computing (EPIC); Probably dying
 - ▶ Intel IA64 (Itanium)

CISC Processor

10

► CISC

- Complex instructions; Large number of instructions; Can complete more complicated functions at instruction level
- Instruction actually invokes microcode. Microcodes are small programs in processor memory
- Slower; Many instructions access memory; varying instruction length; allow no pipelining;

RISC Processor

11

- ▶ No microcode
- ▶ Simple instructions; Fewer instructions; Fast
- ▶ Only load and store instructions access memory
- ▶ Common instruction word length
- ▶ Allows pipelining

Almost all present-day high performance computers use RISC processors

Locality of References

12

► Spatial/Temporal locality

- If processor executes an instruction at time t , it is likely to execute an adjacent/next instruction at $(t + \text{delta}_t)$;
- If processor accesses a memory location/data item x at time t , it is likely to access an adjacent memory location/data item $(x + \text{delta}_x)$ at $(t + \text{delta}_t)$;

Pipelining, Caching and many other techniques all based on the locality of references

Pipelining

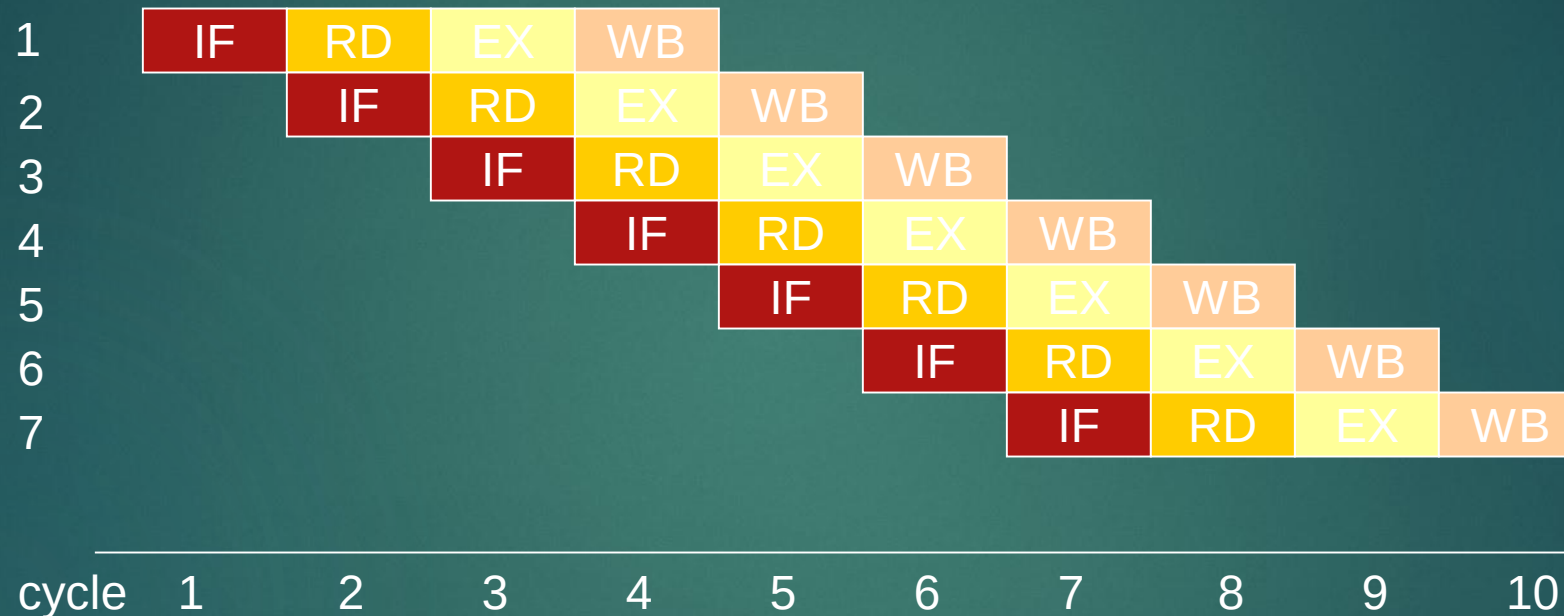
13

- ▶ Overlapping execution of multiple instructions
 - ▶ 1 instruction per cycle
- ▶ Sub-divide instruction into multiple stages; Processor handles different stages of adjacent instructions simultaneously
- ▶ Suppose 4 stages in instruction:
 - ▶ Instruction fetch and decode (IF)
 - ▶ Read data (RD)
 - ▶ Execute (EX)
 - ▶ Write-back results (WB)

Instruction Pipeline

14

instruction



Depth of pipeline: number of stages in an instruction

After the pipeline is full, 1 result per cycle! $CPI = (n + \text{depth} - 1) / n$

With pipeline, 7 instructions take 10 cycles. If no pipeline, 7 instructions take 28 cycles

Inhibitors of Pipelining

15

- ▶ Dependencies between instructions interrupts pipelining, degrading performance
 - ▶ Control dependence.
 - ▶ Data dependence.

Control Dependence

16

- ▶ Branching: when an instruction occurs after an conditional branch; so it is unknown whether that instruction will be executed beforehand
 - ▶ Loop: `for (i=0; i<n; i++) ...; do...enddo`
 - ▶ Jump: `goto ...`
 - ▶ Condition: `if...else...` `if (x>y) n=5;`

Branching in programs interrupts pipeline → degrades performance

Avoid excessive branching!

Data Dependence

17

- ▶ when an instruction depends on data from a previous instruction

```
x = 3*j;  
y = x+5.0; // depends on previous instruction
```


Vector Pipeline

18

- ▶ Vector processors: with vector registers which can hold a vector, e.g. of 128 elements;
 - ▶ Commonly encountered processors are scalar processors, e.g. home PC
- ▶ Efficient for loops involving vectors.

```
for (i=0;i<128;i++)  
    z[i] = x[i] + y[i]
```

Instructions:

```
Vector Load X(1:128)  
Vector Load Y(1:128)  
Vector Add Z=X+Y  
Vector Store Z
```

Vector Pipeline

19

instruction



cycle 1 2 3 ... 133

time

Internal Parallelism

20

- ▶ Functional units: components in processor that actually do the work
 - ▶ Memory operations (MU): load, store;
 - ▶ Integer arithmetic (IU): integer add, bit shift ...
 - ▶ Floating point arithmetic (FPU): floating-point add, multiply ...

Typical instruction latencies

Instruction type	Latency (cycles)
Integer add	1
Floating-point add	3
Floating-point multiply	3
Floating-point divide	31

Division is much slower than add/multiply! Minimize or avoid divisions!

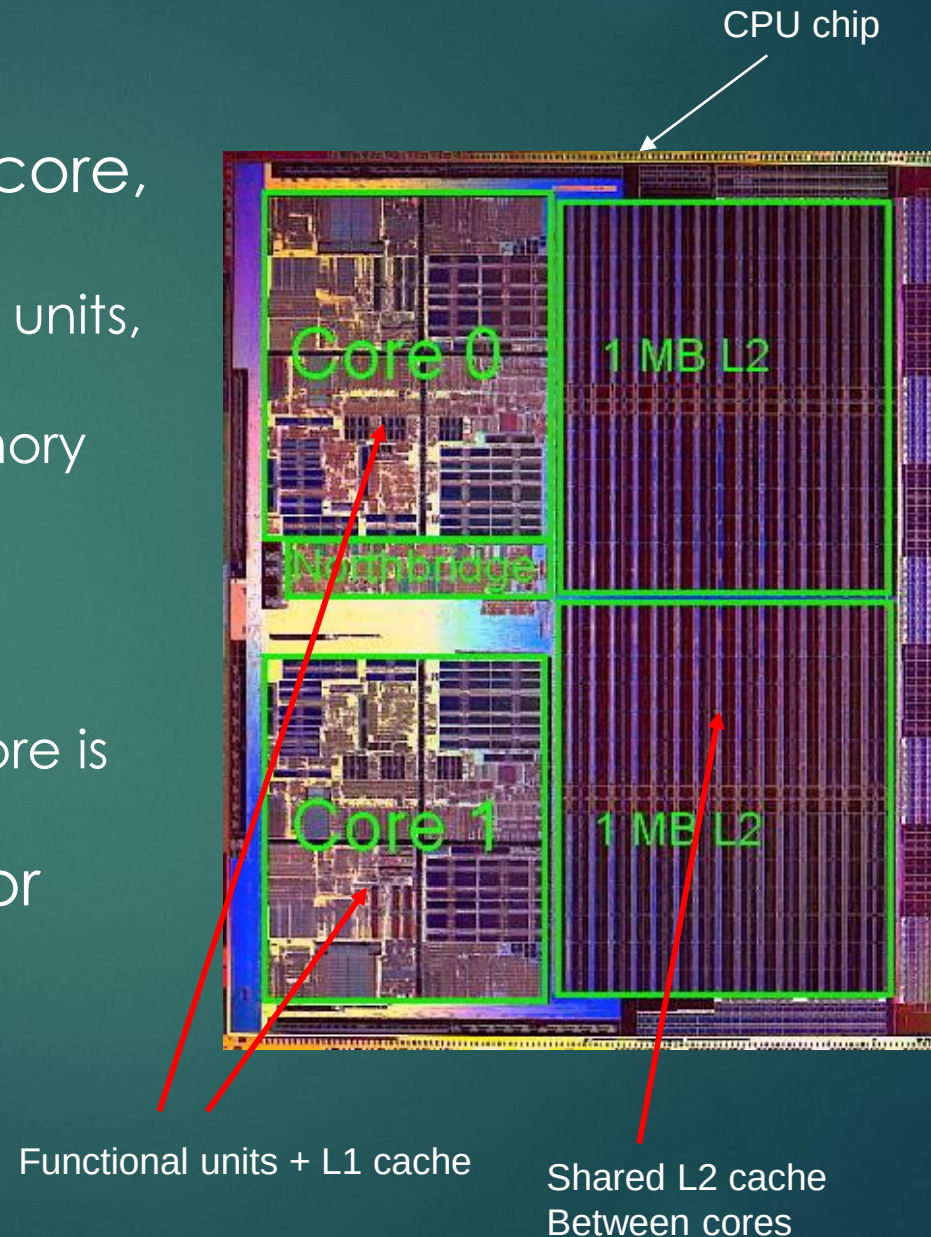
Internal Parallelism

- ▶ Superscalar RISC processors: multiple functional units in processor, e.g. multiple FPUs,
 - ▶ Capable of executing more than one instruction (producing more than one result) per cycle.
 - ▶ Shared registers, L1 cache etc.
- ▶ Need faster memory access to provide data to multiple functional units!
- ▶ Limiting factor: memory-processor bandwidth

Internal Parallelism

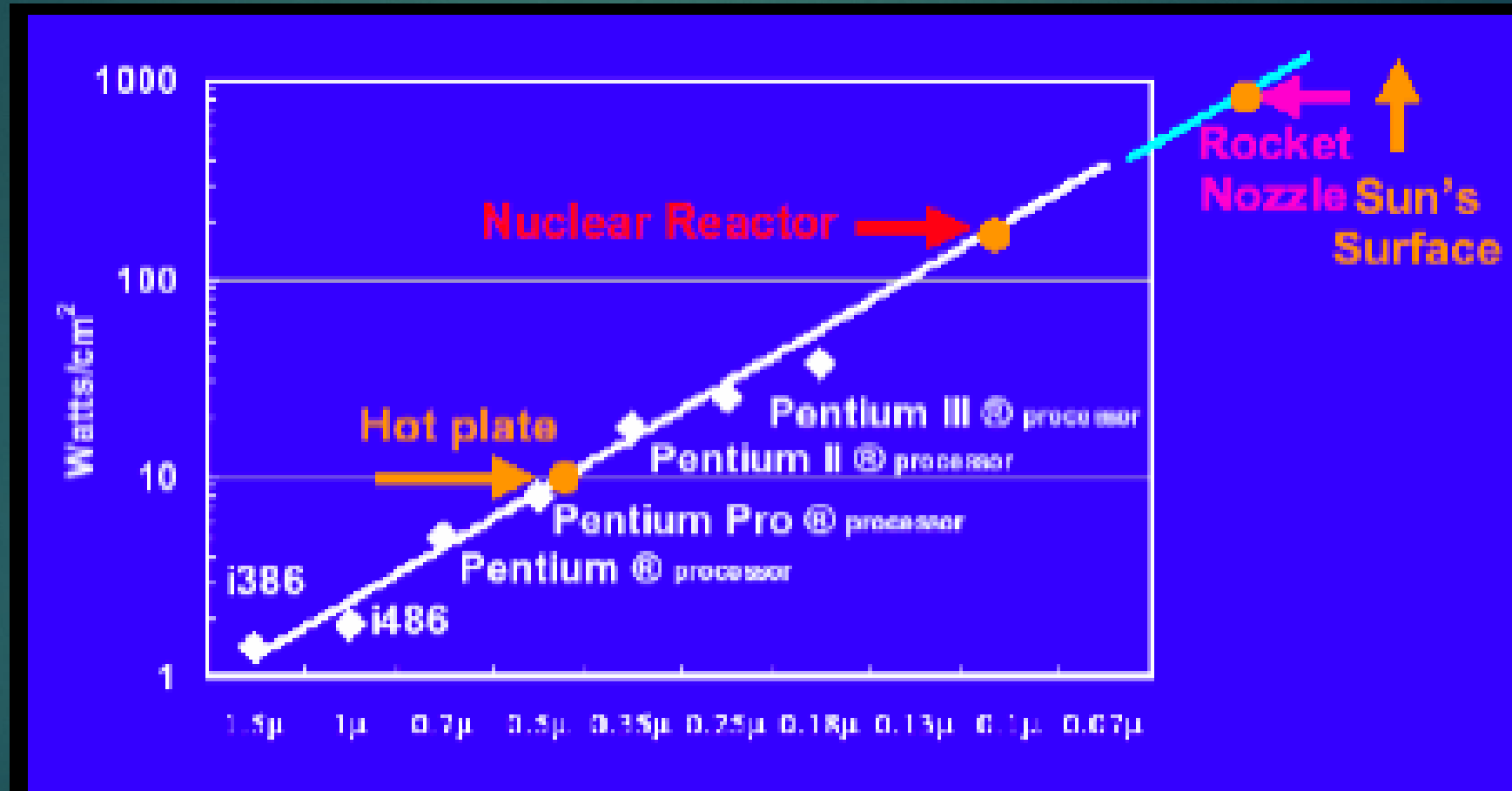
22

- ▶ Multi-core processors: Intel dual-core, quad-core
 - ▶ Multiple execution cores (functional units, registers, L1 cache)
 - ▶ Multiple cores share L2 cache, memory
 - ▶ Lower energy consumption
- ▶ Need **FAST** memory access to provide data to multiple cores
 - ▶ Effective memory bandwidth per core is reduced
- ▶ Limiting factor: memory-processor bandwidth



Heat Flux also Increases with Speed!

23



New Processors are Too Hot!

24



Power Cost of Frequency

- Power $\propto$ Voltage² x Frequency (V^2F)
- Frequency $\propto$ Voltage
- Power $\propto$ Frequency³

	Cores	V	Freq	Perf	Power	PE (Ops/watt)
Superscalar	1	1	1	1	1	1
"New" Superscalar	1X	1.5X	1.5X	1.5X	3.3X	0.45X
Multicore	2X	0.75X	0.75X	1.5X	0.8X	1.88X

(Bigger # is better)
Efficiency

50% more performance with 20% less power

Preferable to use multiple slower devices, than one superfast device

Parallel Processing Architecture

25

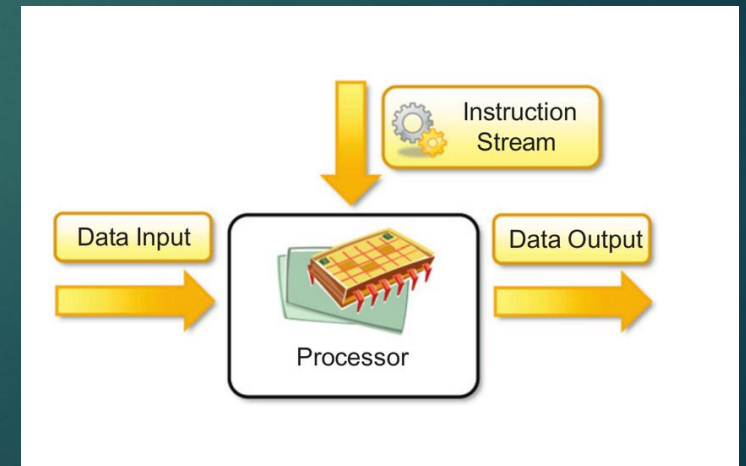
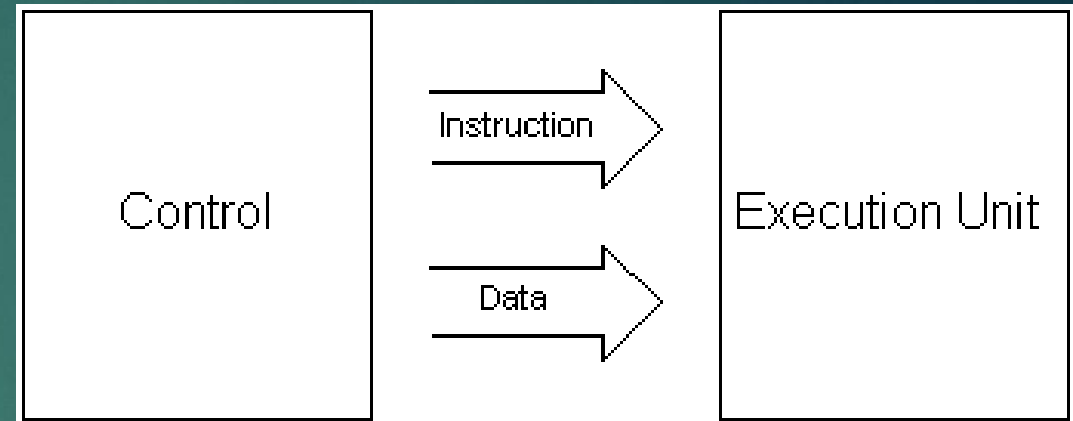
- ▶ Computation Consists of :
 - ▶ Sequential (Stream of) Instructions (operation)
 - ▶ Sequential (Stream of) dataset
- ▶ We can classify into classes of computing system base on their characteristic instructions and dataset:
 - ▶ SISD Single Instruction, Single Data
 - ▶ SIMD Single Instruction, Multiple Data
 - ▶ MISD Multiple Instructions, Single Data
 - ▶ MIMD Multiple Instructions, Multiple Data

		DATA STREAM	
		Single	Multiple
INSTRUCTION STREAM	Single	Single Instruction Single Data SISD	Single Instruction Multiple Data SIMD
	Multiple	Multiple Instruction Single Data MISD	Multiple Instruction Multiple Data MIMD

SISD

Single Instruction, Single Data

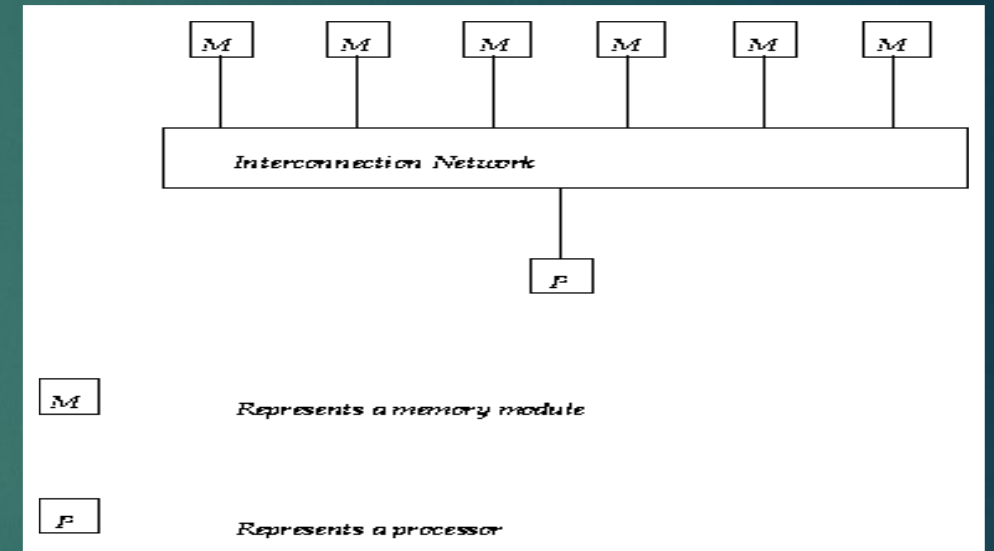
- ▶ Single Instruction Single Data
 - ▶ One stream of instruction
 - ▶ One stream of data
- ▶ Scalar pipeline
 - ▶ To utilize CPU in most of the time
- ▶ Super scalar pipeline
 - ▶ Increase the throughput
 - ▶ Expecting to increase IPC (Instruction Per Cycle) > 1
- ▶ Improvement from increase the “operation frequency”



SISD Bottleneck

27

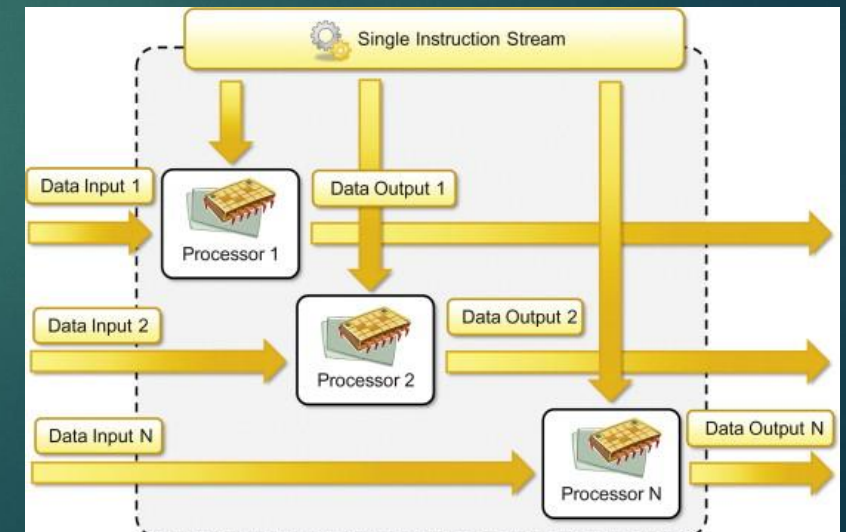
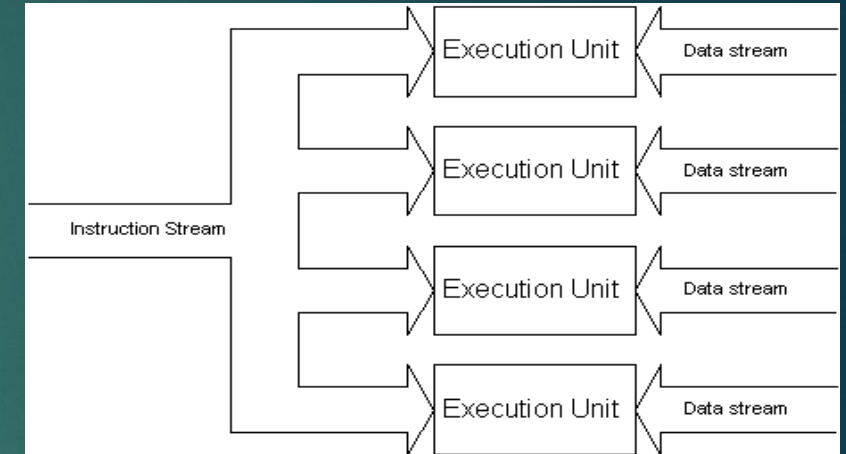
- ▶ Level of Parallelism is low
 - ▶ Data dependency
 - ▶ Control dependency
- ▶ Limitation improvements
 - ▶ Pipeline
 - ▶ Super scalar
 - ▶ Super-pipeline scalar



SIMD

Single Instruction, Multiple Data

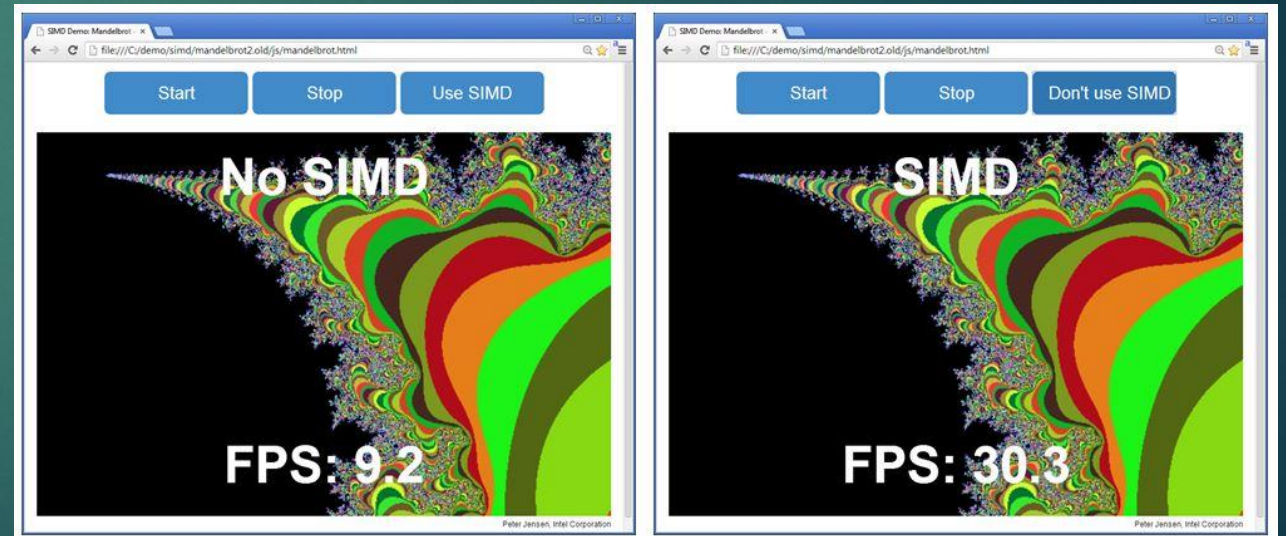
- ▶ Single Instruction, Multiple Data
 - ▶ Single Instruction stream
 - ▶ Multiple data streams
- ▶ Each instruction operate on multiple data in parallel
- ▶ Fine grained Level of Parallelism



SIMD Applications

29

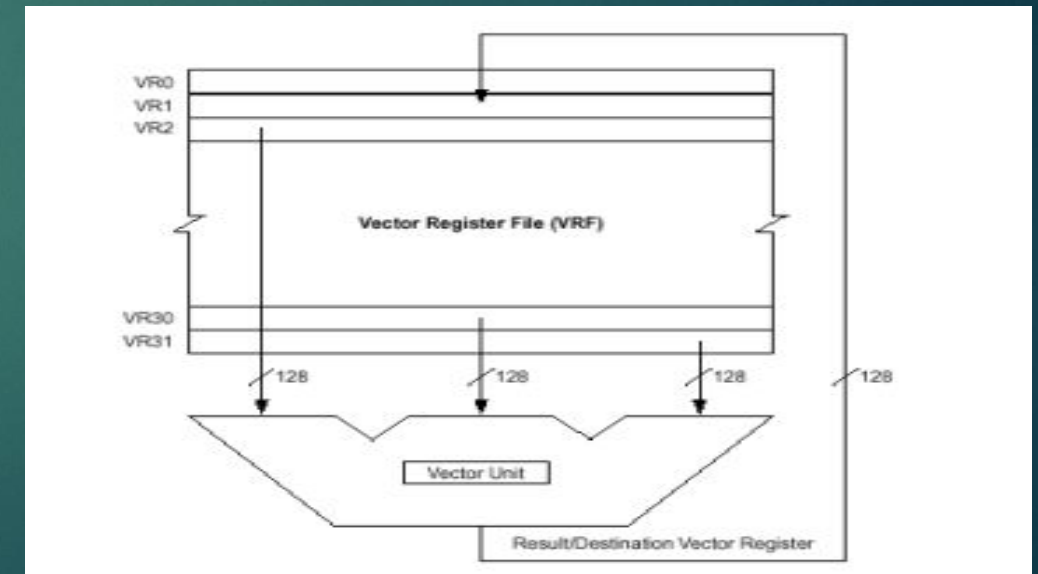
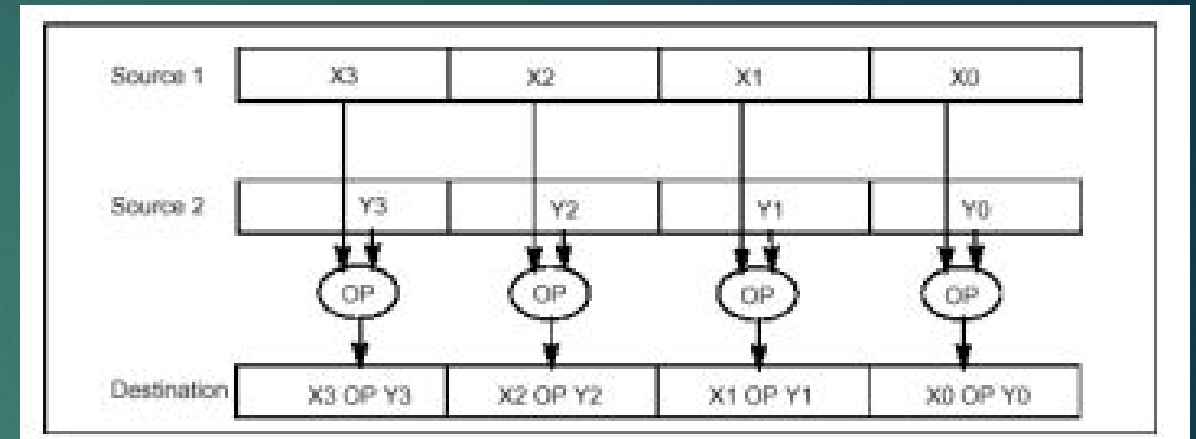
- ▶ A wide variety of applications can be solved by parallel algorithms with SIMD
 - ▶ only problems that can be divided into sub problems, all of those can be solved simultaneously by the same set of instructions
 - ▶ This algorithms are typical easy to implement
- ▶ Examples
 - ▶ Multimedia applications
 - ▶ 2D and 3D image processing
 - ▶ Game and etc
 - ▶ Scientific applications
 - ▶ CAD, Simulations
- ▶ GPU – Graphics Processing Unit



SIMD Operations

30

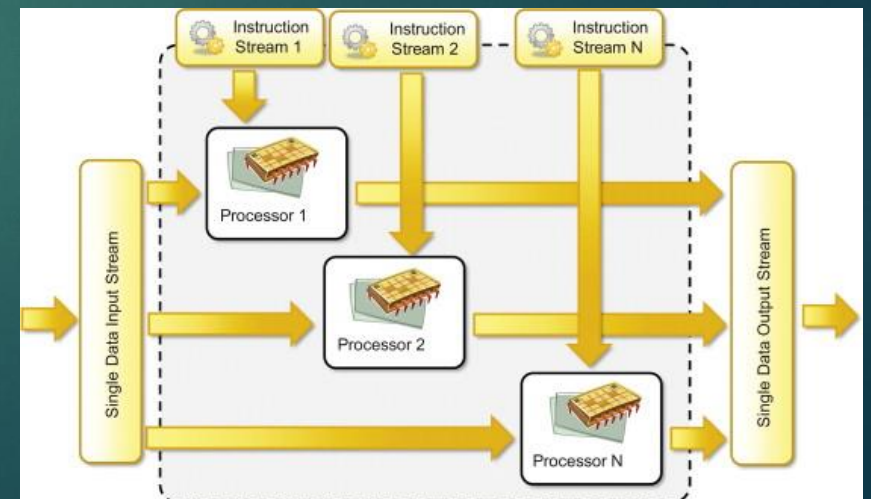
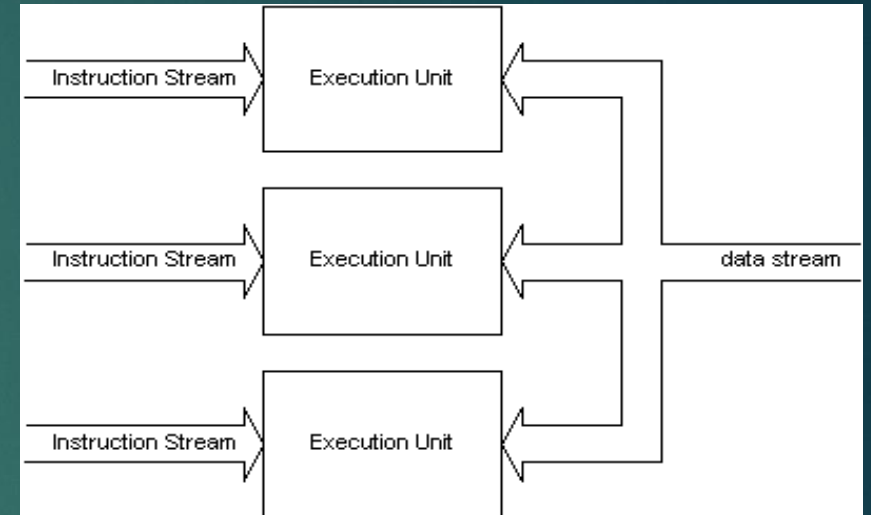
- ▶ SIMD instructions supports
 - ▶ Load and store
 - ▶ Integer
 - ▶ Floating point
 - ▶ Logical and Arithmetic instructions
 - ▶ Additional instruction (optional)
 - ▶ Cache instructions to support different locality for different type of application characteristic



MISD

Multiple Instructions, Single Data

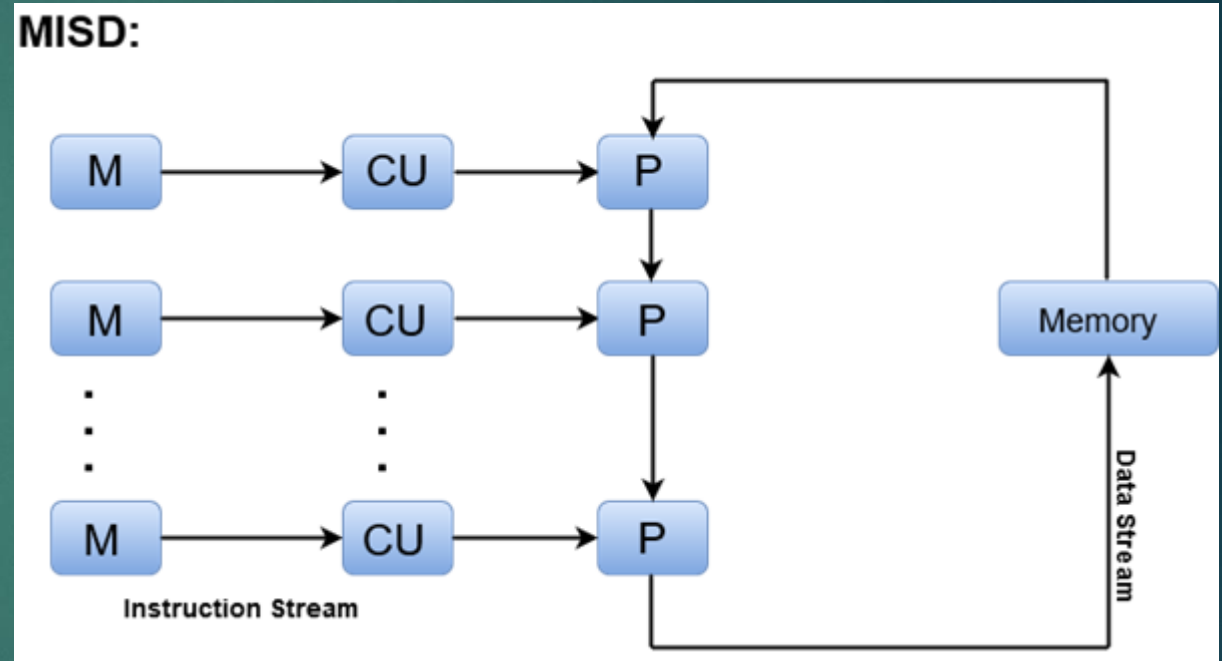
- ▶ Multiple Instructions Single Data
 - ▶ Multiple streams of instruction
 - ▶ Single stream of data
 - ▶ Multiple functionally unit operate on single data
 - ▶ Possible list of instructions or a complex instruction per operand (CISC)
- ▶ Receive less attention compare to the other



MISD Bottleneck

32

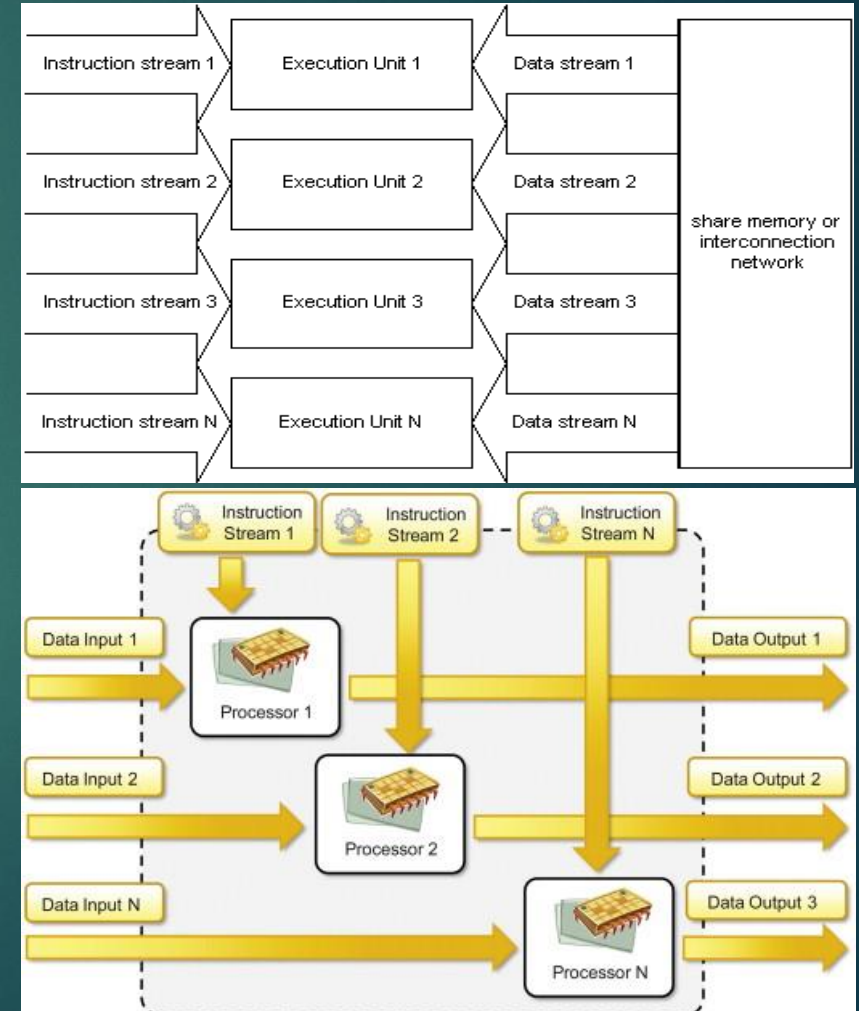
- ▶ Low level of parallelism
- ▶ High synchronizations
- ▶ High bandwidth required
- ▶ CISC bottleneck
- ▶ High complexity



MIMD

Multiple Instructions, Multiple Data

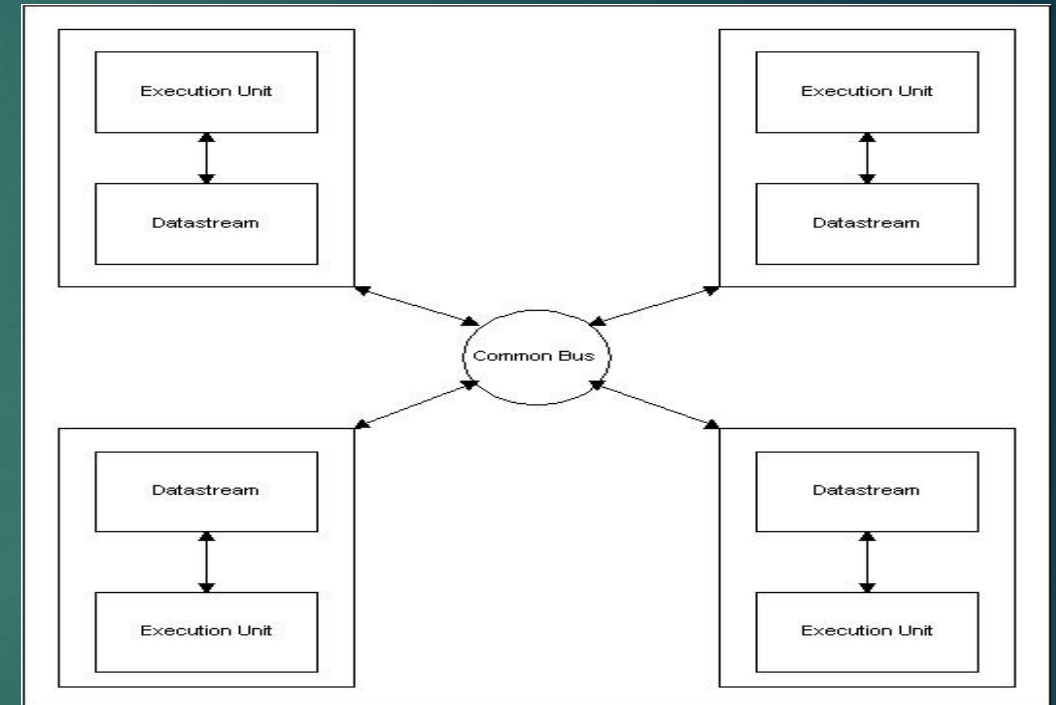
- ▶ Multiple Instruction Multiple Data
 - ▶ Multiple streams of instructions
 - ▶ Multiple streams of data
- ▶ Middle grained Parallelism level
- ▶ Used to solve problem in parallel are those problems that lack the regular structure required by the SIMD model.
- ▶ Implements in cluster or SMP systems
- ▶ Each execution unit operate asynchronously on their own set of instructions and data, those could be a sub-problems of a single problem.



MIMD

34

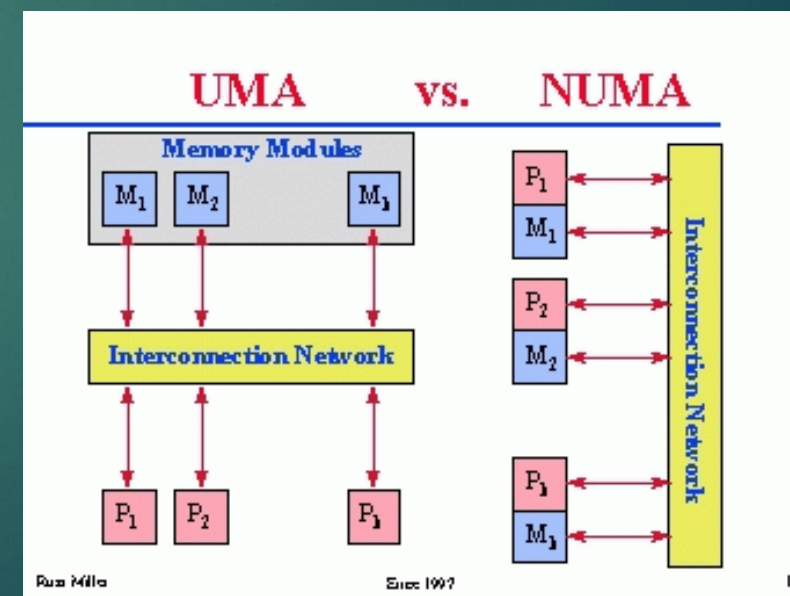
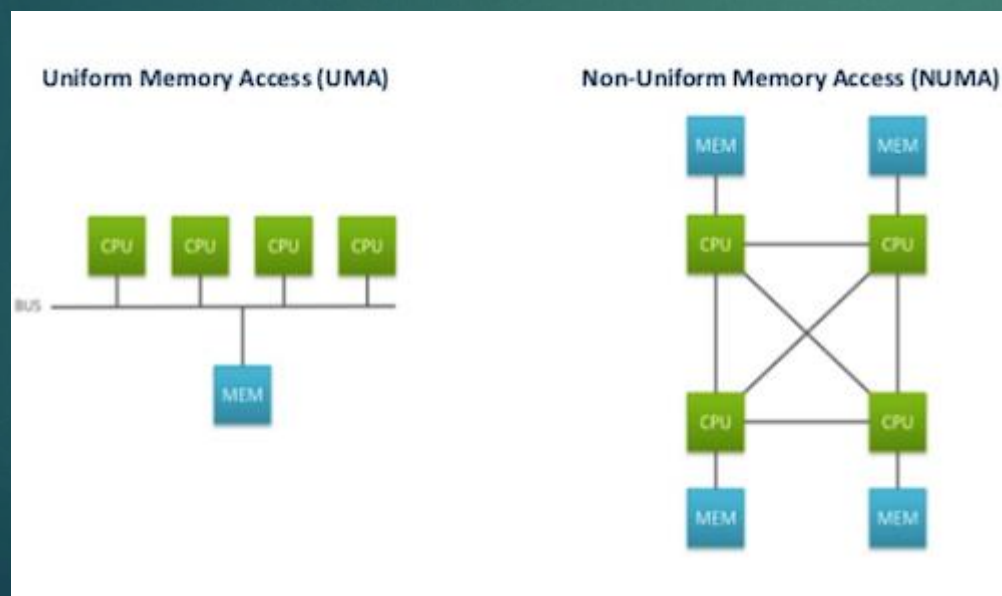
- ▶ Requires
 - ▶ Synchronization
 - ▶ Inter-process communications
 - ▶ Parallel algorithms
 - ▶ Those algorithms are difficult to design, analyse and implement



MIMD System Architectures

35

- ▶ A systems may also be classified by their architectures.
 - Uniform Memory Access (UMA)
 - Non-Uniform Memory Access (NUMA)



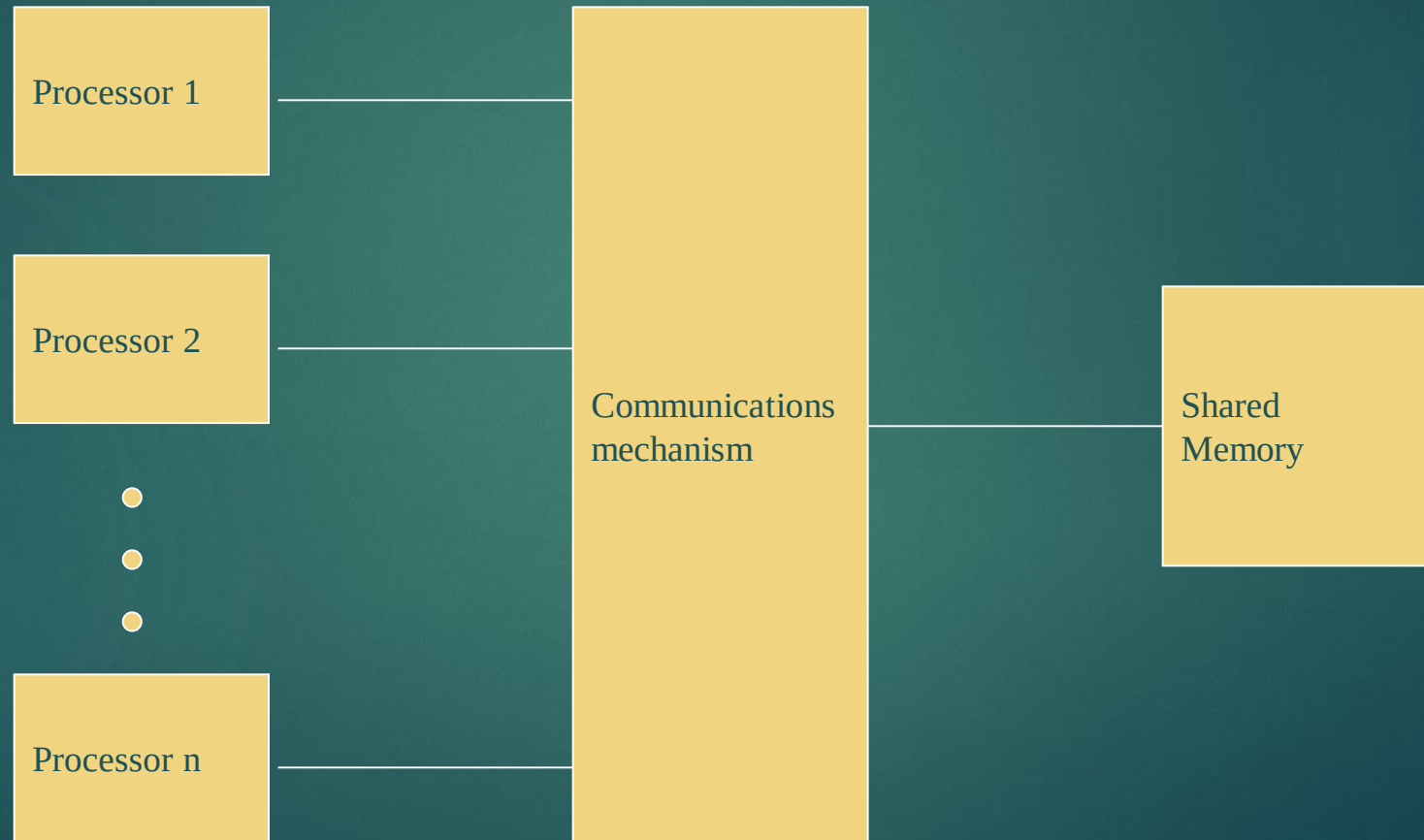
Uniform memory access (UMA)

36

- ▶ The UMA is a type of symmetric multiprocessor, or SMP, that has two or more processors that perform symmetric functions. UMA gives all CPUs equal (uniform) access to all memory locations in shared memory. They interact with shared memory by some communications mechanism like a simple bus or a complex multistage interconnection network.

Uniform memory access (UMA) Architecture

37

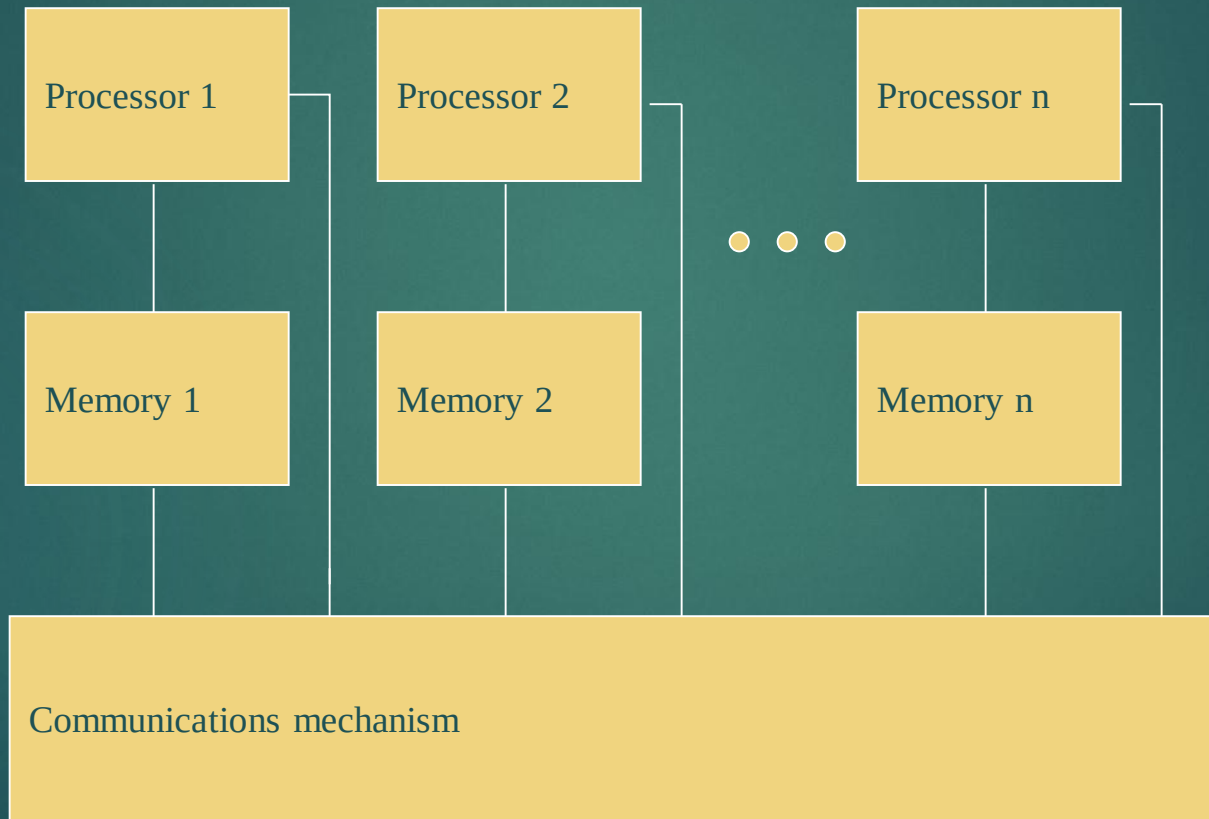


Nonuniform memory access (NUMA)

- ▶ NUMA architectures, unlike UMA architectures do not allow uniform access to all shared memory locations. This architecture still allows all processors to access all shared memory locations but in a nonuniform way, each processor can access its local shared memory more quickly than the other memory modules not next to it.

Nonuniform memory access (NUMA) Architecture

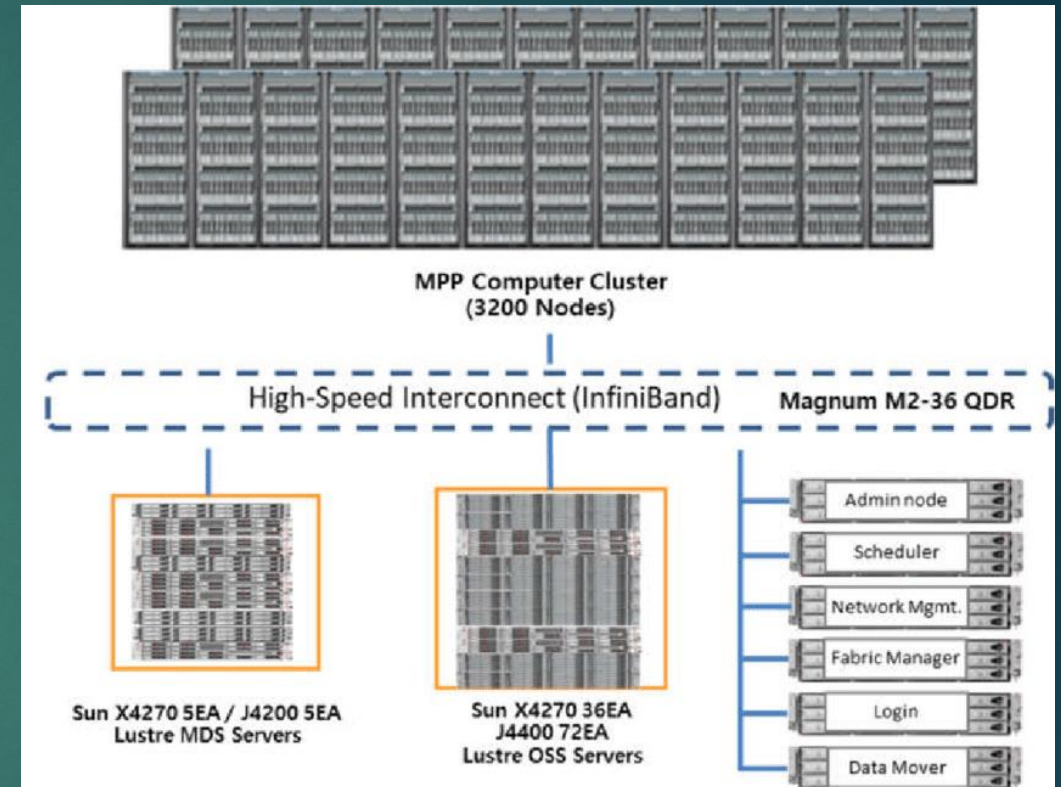
39



MPP Super-Computer

40

- ▶ Massively Parallel Processing
- ▶ High performance of single processor
- ▶ Multi-processor MP
- ▶ Cluster Network
- ▶ Mixture of everything

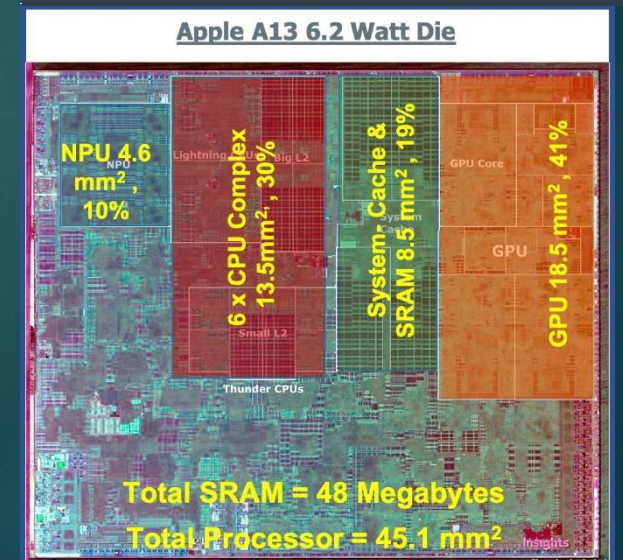
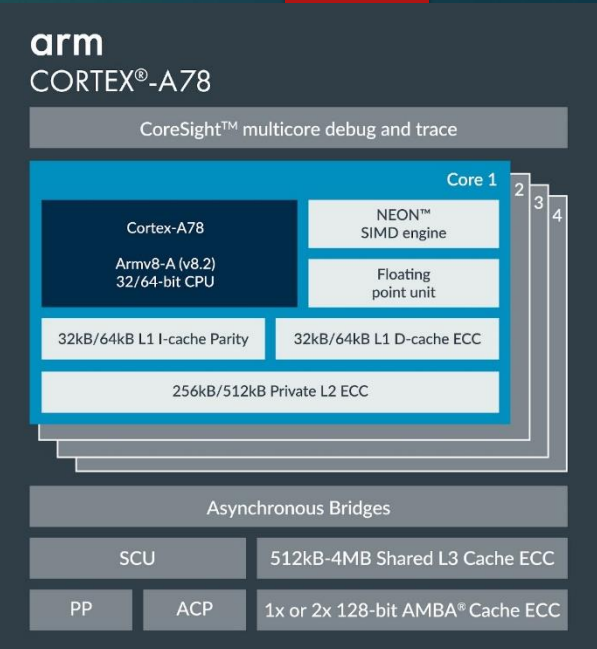


- ▶ Cluster of High performance MP nodes

High Performance Computing in Processor

41

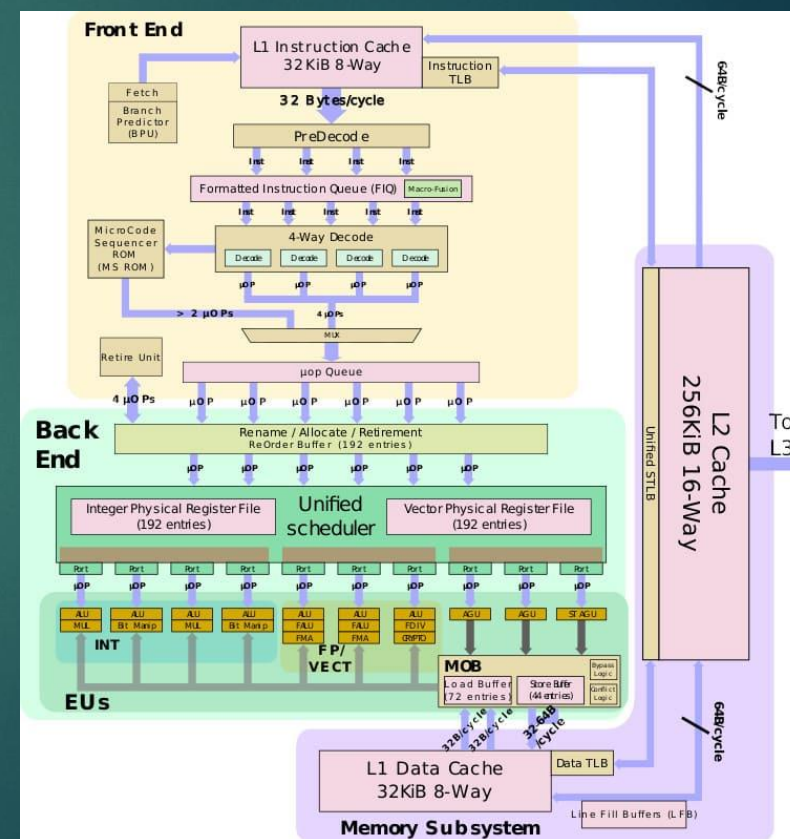
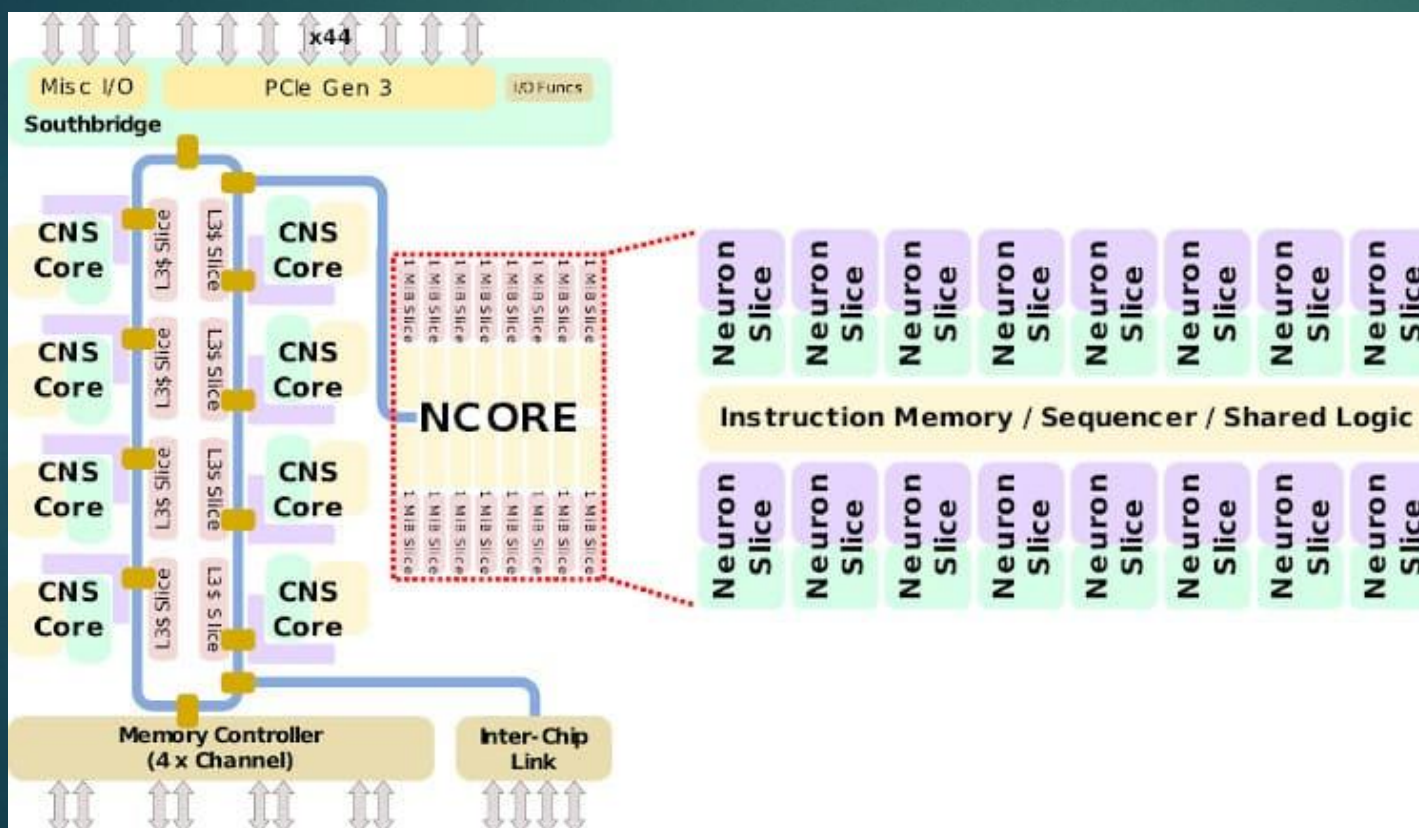
- ▶ Performance for Single Processor
 - ▶ Clock => Pipeline => Local Access
- ▶ Multiple Processor
 - ▶ Multi-Core
 - ▶ Multi-Processor / Multi-Computer
- ▶ Accelerators => Internal Processing Unit
 - ▶ External vs Internal => Local for Speed
 - ▶ FPU, VPU, GPU, NPU, etc.



High Performance Computing in Processor

42

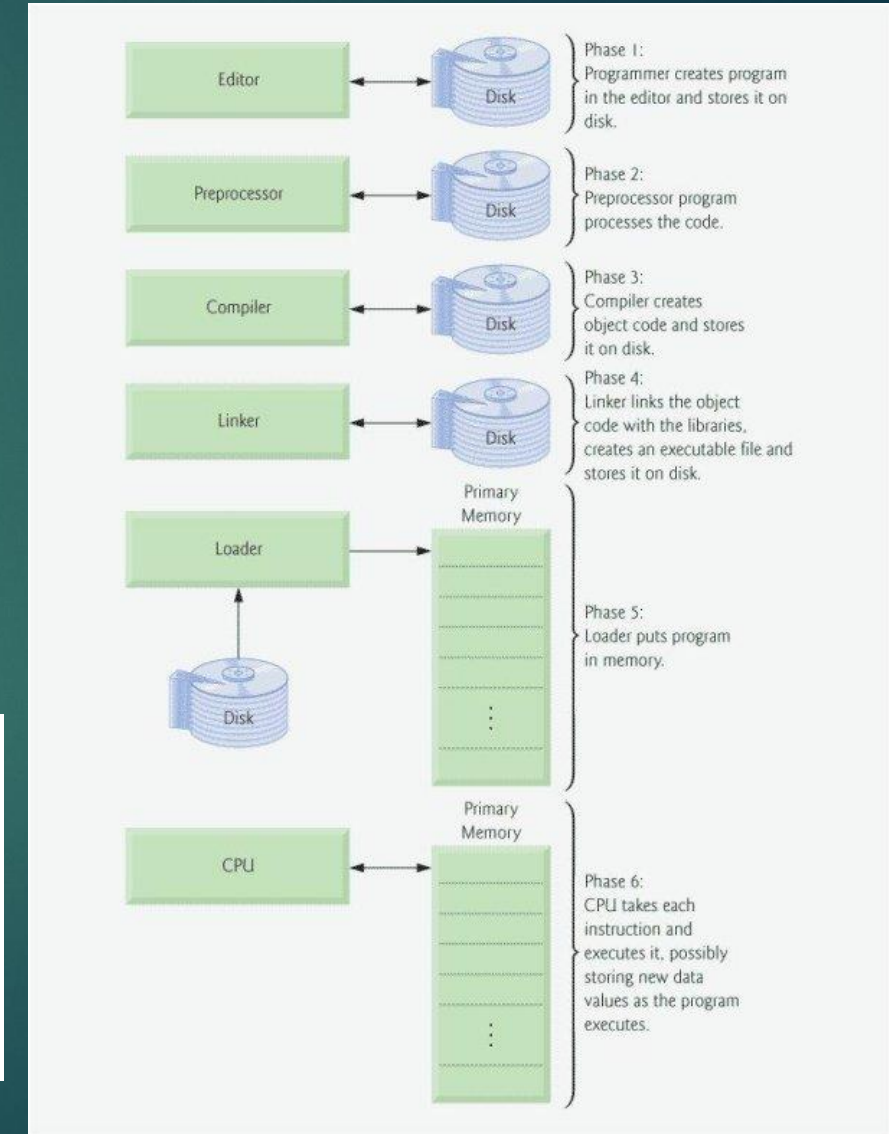
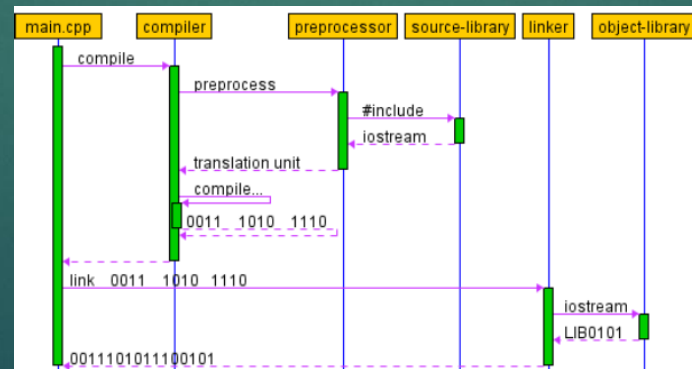
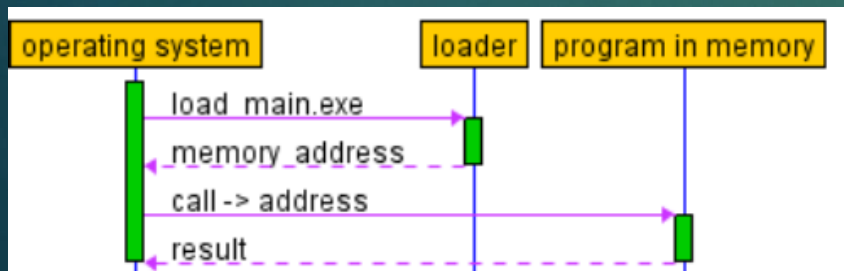
- Increase number of executing units => More Logic



CPU-Based Code Development and Execution

43

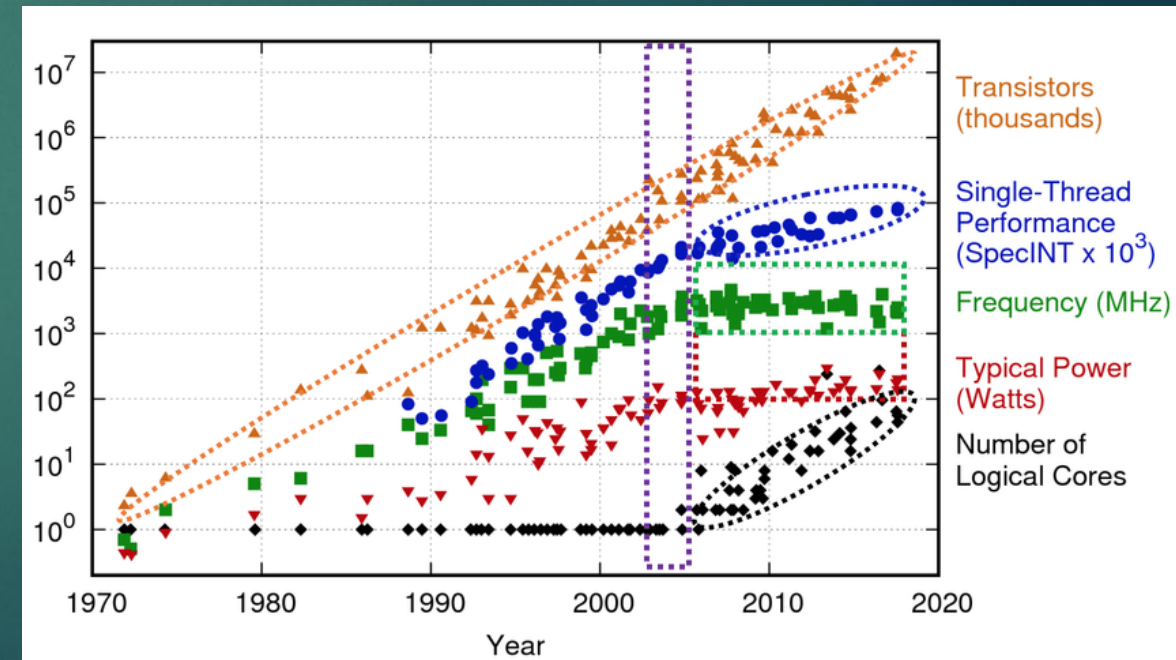
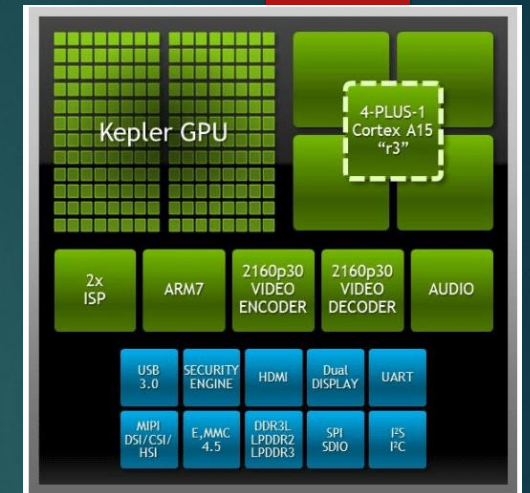
- ▶ Source Code => Edit
- ▶ Compiler => Object Code
- ▶ Linker => Executable Code
- ▶ Executable Code => Loader => Memory
- ▶ Execute Codes / Instructions



Conclusion

44

- ▶ CPU / Processor => Control and Computing
- ▶ Increasing Performance by
 - ▶ Clock Speed
 - ▶ Parallel Executing
 - ▶ Pipeline
 - ▶ Multi-Core / Multi-Processor
 - ▶ Dedicated Units
- ▶ Parallel Executing needs more Resources/Hardware/Logics



END

QUESTIONS & ANSWERS